

Data Structure and Algorithms



Objectives



- Fundamental Data Structures
 - Arrays
 - ArrayLists
 - Queues
 - Stacks
 - LinkedLists

DS types

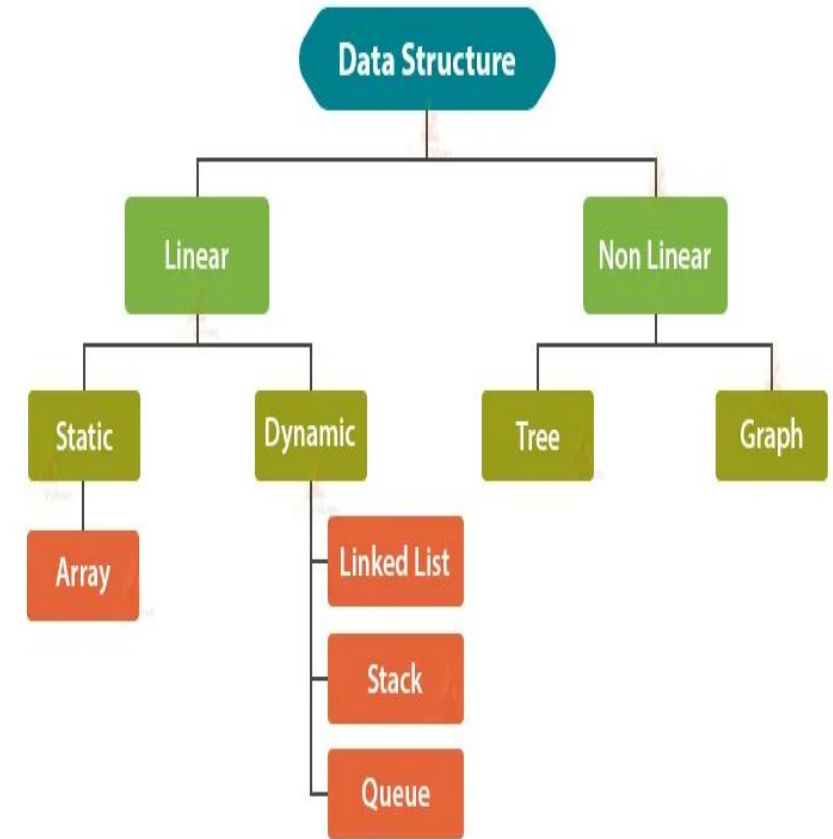
- Based on the organization of elements,

Linear Data Structures –

Elements are accessed in a sequential order

Non-linear Data Structures –

- Elements are accessed in a non-linear order





Arrays

- An array stores a collection of **identically typed** entities
- The **array** is the most commonly used data storage structure; it's built into most programming languages.



- An array is a fixed-size, ordered collection of elements of the same data type.
- It allows you to store **multiple** values of the same type under a **single variable** name.
- Each element in an array is accessed by its index, which starts from 0 and goes up to the array's length minus one.

- An array refers to a data structure that contains **homogeneous** elements

40	55	23	87	21	11	94	Elements
0	1	2	3	4	5	6	Index

- This is an array of seven elements. All the elements are integers and homogeneous.
- The green box below the array is called the index, which always starts from zero and goes up to $n-1$ elements.
- In this case, as there are seven elements, the index is from zero to six.



There are 3 main features of an array:

- **Dynamic allocation:** In arrays, the memory is created dynamically, which reduces the amount of storage required for the code.
- **Elements stored under a single name:** All the elements are stored under one name. This name is used any time we use an array.
- **Occupies contiguous location:** The elements in the arrays are stored at adjacent positions. This makes it easy for the user to find the locations of its elements.

Basic terminologies of array

- **Array Index:**

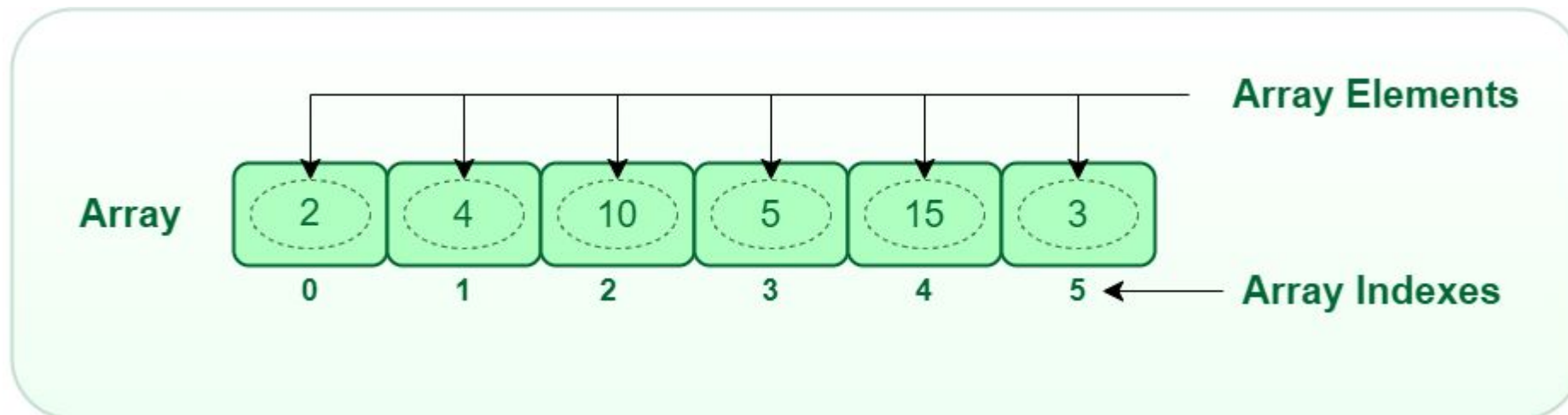
In an array, elements are identified by their indexes. Array index starts from 0.

- **Array element:**

Elements are items stored in an array and can be accessed by their index.

- **Array Length:**

The length of an array is determined by the number of elements it can contain.



```
String[] cars;
```



Declaration and Initialization syntax

```
data_type[] variable_name;
```

E.g. `int[] numbers;`

..... •

```
data_type variable_name[];
```

E.g. `int numbers[];`



Example

```
string[] names = {"John", "Mary", "David"};
```

In this example, we declare an array of strings named **names** and initialize it with the values "John", "Mary", and "David".



Example

```
// Declaration and initialization with specified values  
int[] numbers = {1, 2, 3, 4, 5};
```

```
string[] favPlayers =  
    {"Zidane", "Maradona", "Pele"};
```



To declare arrays with default values

```
data_type[] variable_name = new data_type[size];
```

```
int[] numbers = new int[5];
```

In this example, we declare an array of integers named **numbers** with a length of 5.

Since we haven't specified any values, each element in the array is initialized with the default value of **0**.

The `new` keyword indicates that a new array is being created.



```
// Declaration with specified size and then  
initialization
```

```
int[] numbers = new int[5];  
numbers[0] = 1;  
numbers[1] = 2;  
numbers[2] = 3;  
numbers[3] = 4;  
numbers[4] = 5;
```



```
boolean[] flags = new boolean[3];
```

In this example, we declare an array of booleans named **flags** with a length of **3**. Since we haven't specified any values, each element in the array is initialized with the default value of **false**.



Example

```
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};
```

```
int[] myNum = {2, 4, 10, 5, 15, 3};
```



Default values of data types

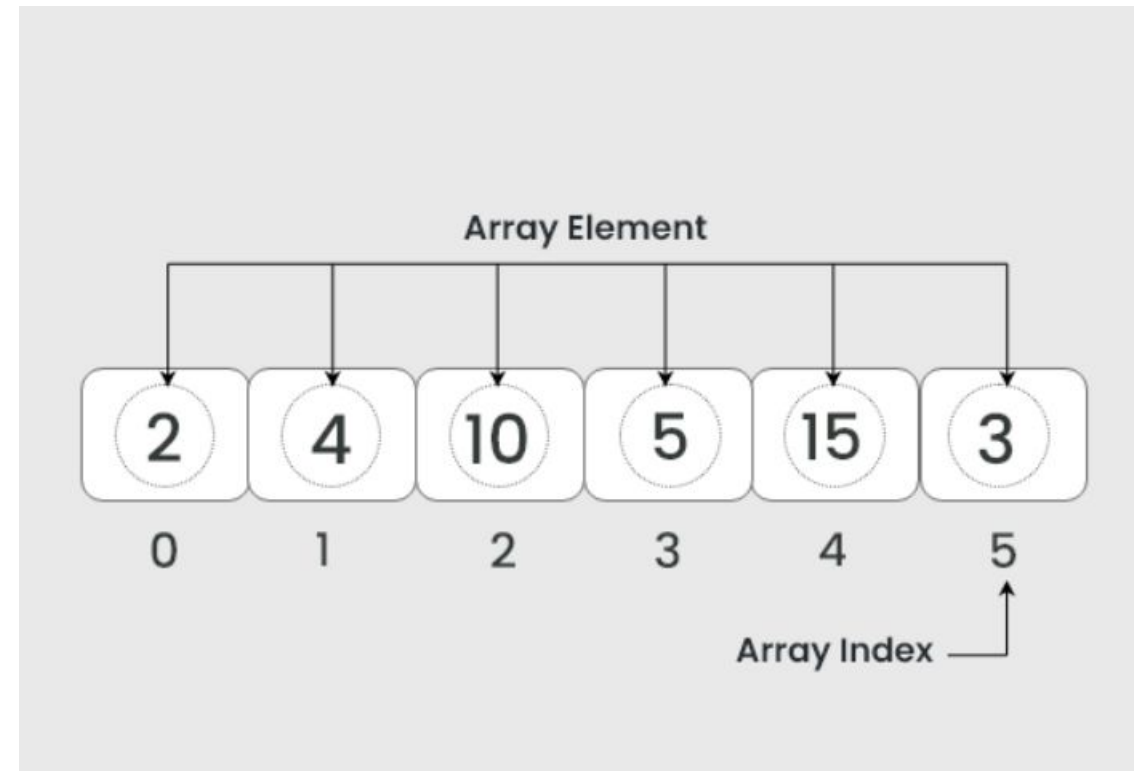
Type	Default Value
boolean	false
byte, short, int, long	0
float, double	0.0
char	'\u0000'
Reference Type	null

Accessing Elements

- Just use indexing
- Array elements are accessed using an index number in square brackets

// access array elements

array[index]





Time complexity

- Accessing an element in an array involves retrieving the value stored at a specific index position within the array.
- The time complexity for accessing an array element is constant, **$O(1)$** , regardless of the size of the array. This is because arrays offer direct access to elements based on their indices, and accessing any element requires a single operation, regardless of the size of the array.



```
public class ArrayAccessExample {
    public static void main(String[] args) {
        int[] array = {10, 20, 30, 40, 50}; // Creating an array

        // Accessing the first element (index 0)
        int firstElement = array[0];
        System.out.println("First element: " + firstElement);

        // Accessing the third element (index 2)
        int thirdElement = array[2];
        System.out.println("Third element: " + thirdElement);

        // Accessing the last element (index 4)
        int lastElement = array[array.length - 1];
        System.out.println("Last element: " + lastElement);
    }
}
```



Regardless of the size of the array, accessing an element involves directly referencing its index position, resulting in constant time and space complexity.

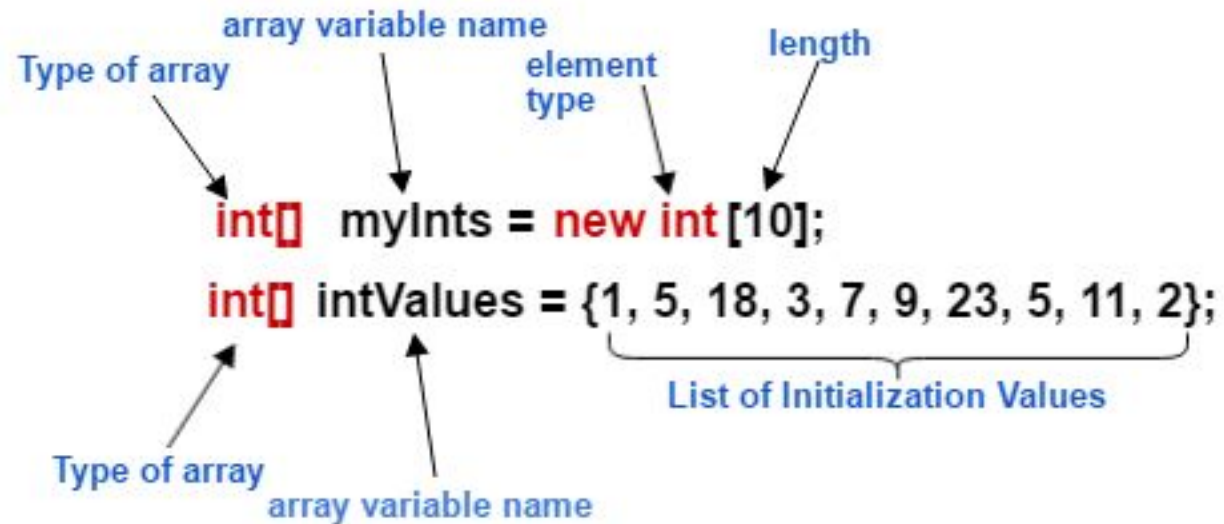
Array Operations

- **Traverse** – print all the array elements one by one.
- **Insertion** – Adds an element at the given index.
- **Deletion** – Deletes an element at the given index.
- **Search** – Searches an element using the given index or by the value.
- **Update** – Updates an element at the given index.
- **Display** – Displays the contents of the array.

Length of an array

```
int[] intValues = {1, 5, 18, 3, 7, 3};
```

```
intValues.length
```





length property

```
public class Arrays {  
    public static void main(String[] args) {  
        int[] numbers = {10, 20, 30, 40, 50};  
        System.out.println(numbers.length);  
    }  
}
```



Traverse

```
public class Arrays {  
    public static void main(String[] args) {  
        int[] num = {10, 20, 30, 40, 50};  
        for (int i = 0; i < num.length; i++) {  
            System.out.println(num[i]);  
        }  
    }  
}
```



Time complexity of traversing an array

- Traversing an array means visiting each element of the array exactly once. This operation is essential for various tasks, such as searching, sorting, or performing any operation on array elements.
- The time complexity of traversing an array is linear, **$O(n)$** , where 'n' is the number of elements in the array.



Let's break down why traversing an array has linear time complexity:

- **Sequential Access:** Arrays provide contiguous memory allocation. Accessing elements in an array sequentially involves visiting each element one by one, starting from the first element and ending at the last. Since each element is visited exactly once, the time taken is directly proportional to the number of elements in the array.
- **Constant Time Access:** Accessing individual elements in an array using their index has constant time complexity, $O(1)$. However, when traversing the entire array, each element must be accessed, resulting in a linear increase in time as the array size grows.
- **Loop Iteration:** Traversing an array typically involves using a loop (e.g., for loop) to iterate over each element. The loop runs 'n' times, where 'n' is the size of the array. Therefore, the time complexity of the traversal operation becomes $O(n)$, as the loop iterates once for each element in the array.



Searching an element from an array



insertion

- Insertion at the end (appending)
- Insertion at the start
- Insertion at any index

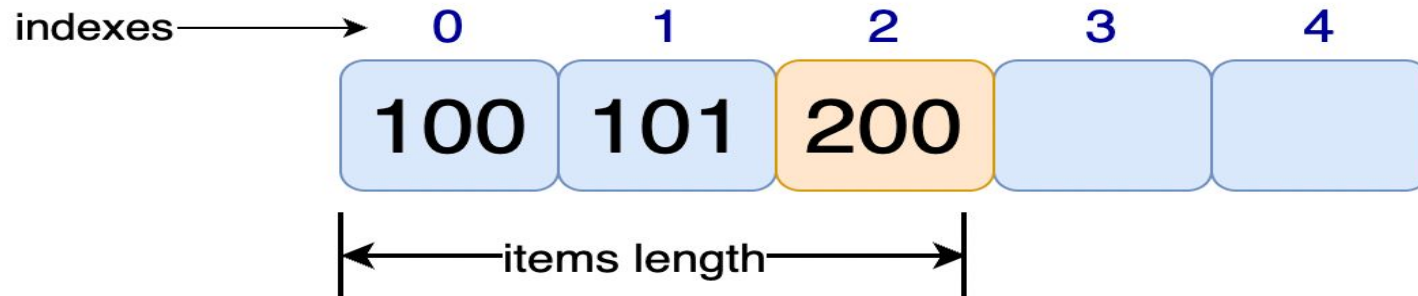
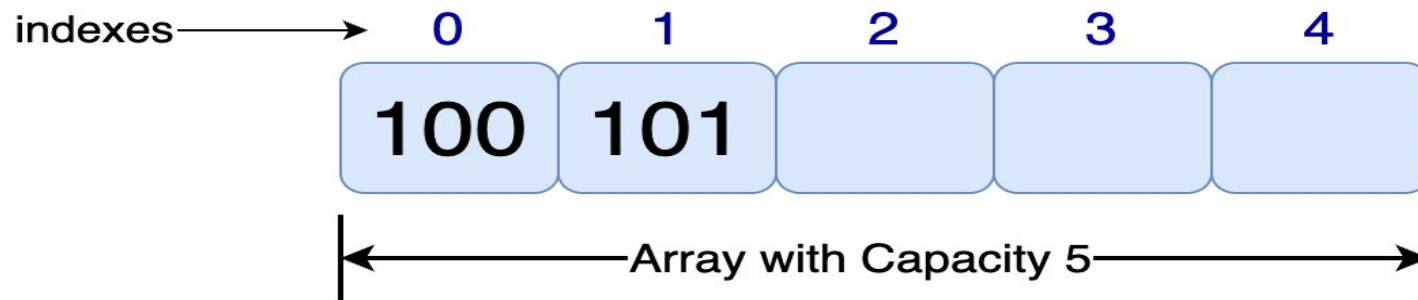


Insertion at the end (when space is available)

- If the array has space available at the end (i.e., the current size is less than the capacity of the array), inserting an element at the end involves the following steps:
 1. Find the index where the new element should be inserted (which is the current size of the array).
 2. Assign the new element to that index.

Since these steps involve only **constant-time** operations (finding the index and assigning the element), the time complexity for this scenario is **$O(1)$** .

```
arr[arr.length - 1] = element;
```





```
public static int[] appendIfSpacious(int arr[], int element) {  
    arr[arr.length - 1] = element;  
    return arr;  
}
```

The time complexity of this function is $O(1)$.

Accessing the last element of the array (`arr[arr.length - 1]`) takes constant time because it involves a single arithmetic operation and accessing an array element by index, both of which are constant-time operations. Assigning the new element also takes constant time.



```
public class AppendingIfRoomy {
    public static int[] appendIfSpacious(int arr[], int elemnt) {
        arr[arr.length - 1] = element;
        return arr; }
    public static void main(String[] args) {
        int[] evens = new int[4];
        evens[0] = 2;
        evens[1] = 4;
        evens[2] = 6;
        appendIfSpacious(evens, 8);
        System.out.println(Arrays.toString(evens));
    }
}
```

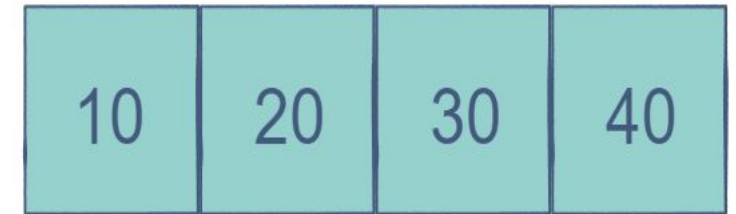
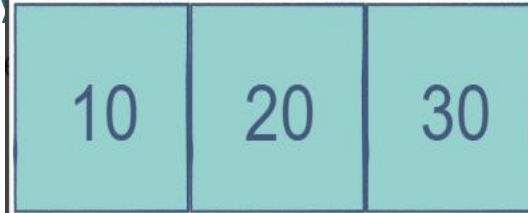


In this example, if the array has space available at the end, inserting the new element is an $O(1)$ operation

Appending(insertion at the end when no extra slot)

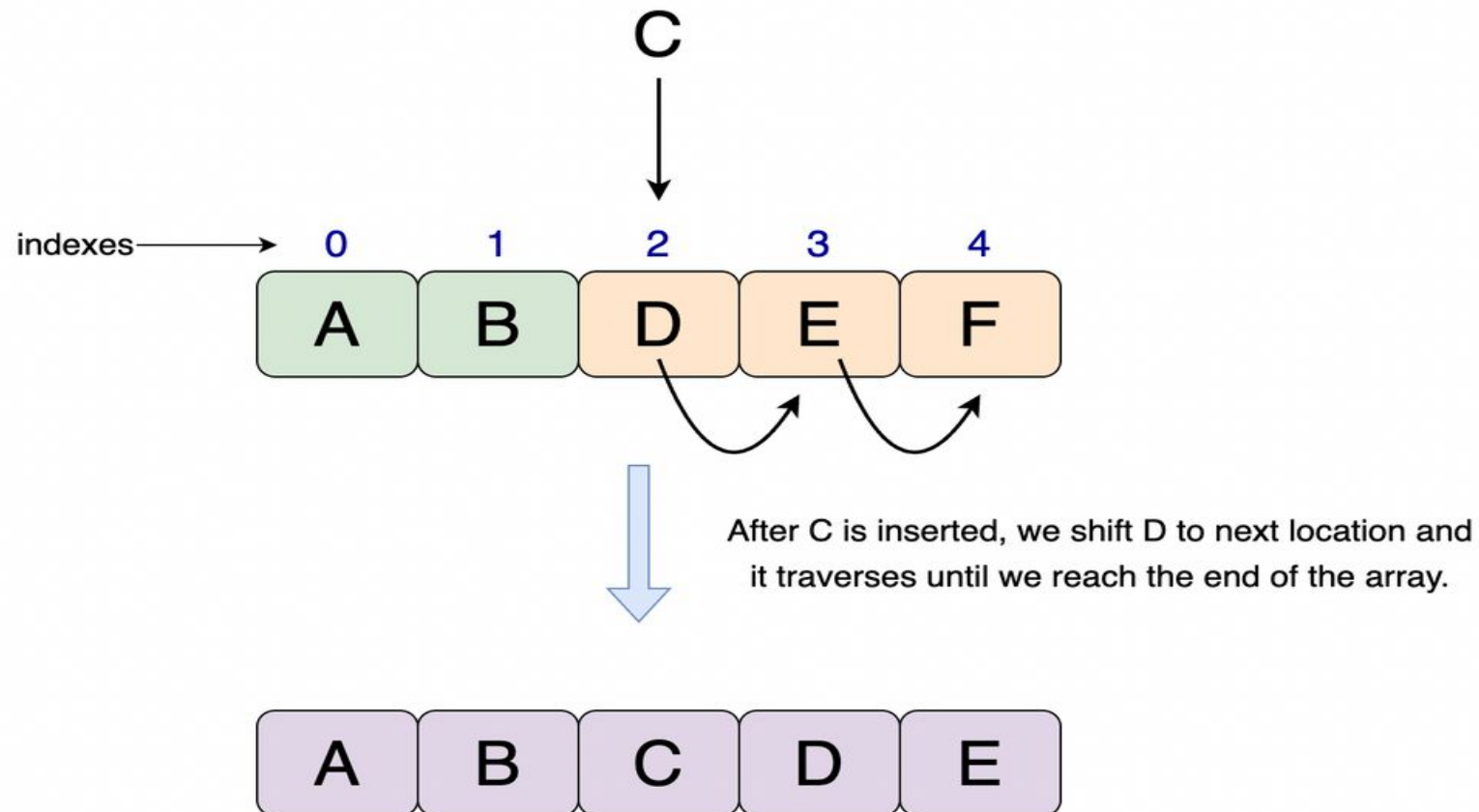
- In Java, arrays can't grow once they have been created. so to add an element you need to:

1. Create a new, larger array.
2. Copy over the content of the original array.
3. Insert the new element at the end.



Insertion at the end (when resizing is required):

- Copying 'n' elements to the new array takes $O(n)$ time, where 'n' is the number of elements in the array. Therefore, the time complexity for this scenario is $O(n)$.





```
public int[] appendElement(int arr[], int element) {  
    int newArray[] = new int[arr.length + 1];  
  
    for (int i = 0; i < arr.length; i++) {  
        newArray[i] = arr[i];  
    }  
  
    newArr[arr.length] = element;  
  
    return newArray;  
}
```



```
public class AppendingElementsAtArrays {
    public static int[] appendElement(int arr[], int element) {

        int newArr[] = new int[arr.length + 1];

        for (int i = 0; i < arr.length; i++) {
            newArr[i] = arr[i];
        }
        newArr[arr.length] = element;

        return newArr;
    }
    public static void main(String[] args) {
        int[] numbers = {2, 4, 6};
        int[] newarr = appendElement(numbers, 8);

        System.out.println(Arrays.toString(newarr));
    }
}
```



Time complexity

- **Copying Existing Elements:** The loop iterates over each element of the input array **arr** and copies it to the new array **newArray**. This loop runs **n** times, where **n** is the length of the input array **arr**.

Time Complexity: $O(n)$

- **Appending the New Element:** After copying existing elements, the function appends the new element to the end of the new array.

Appending the new element takes constant time, $O(1)$.

- Overall, the time complexity of the function is dominated by the loop that copies existing elements, resulting in a time complexity of **$O(n)$** .



Insert at the start

```
public static int[] insertAtStart(int arr[], int element) {  
    int newArr[] = new int[arr.length + 1];  
  
    newArr[0] = element;  
  
    for(int i = 0; i < arr.length; i++) {  
        newArr[i + 1] = arr[i];  
    }  
    return newArr;  
}
```



Time Complexity

Creating New Array: The function creates a new array **newArr** with a length of **arr.length + 1**, where **arr.length** is the length of the input array **arr**.

Creating the new array takes **constant time, $O(1)$** .

Inserting the New Element: The function inserts the new element at the beginning of the new array.

Inserting the new element at the beginning of the array takes **constant time, $O(1)$** .

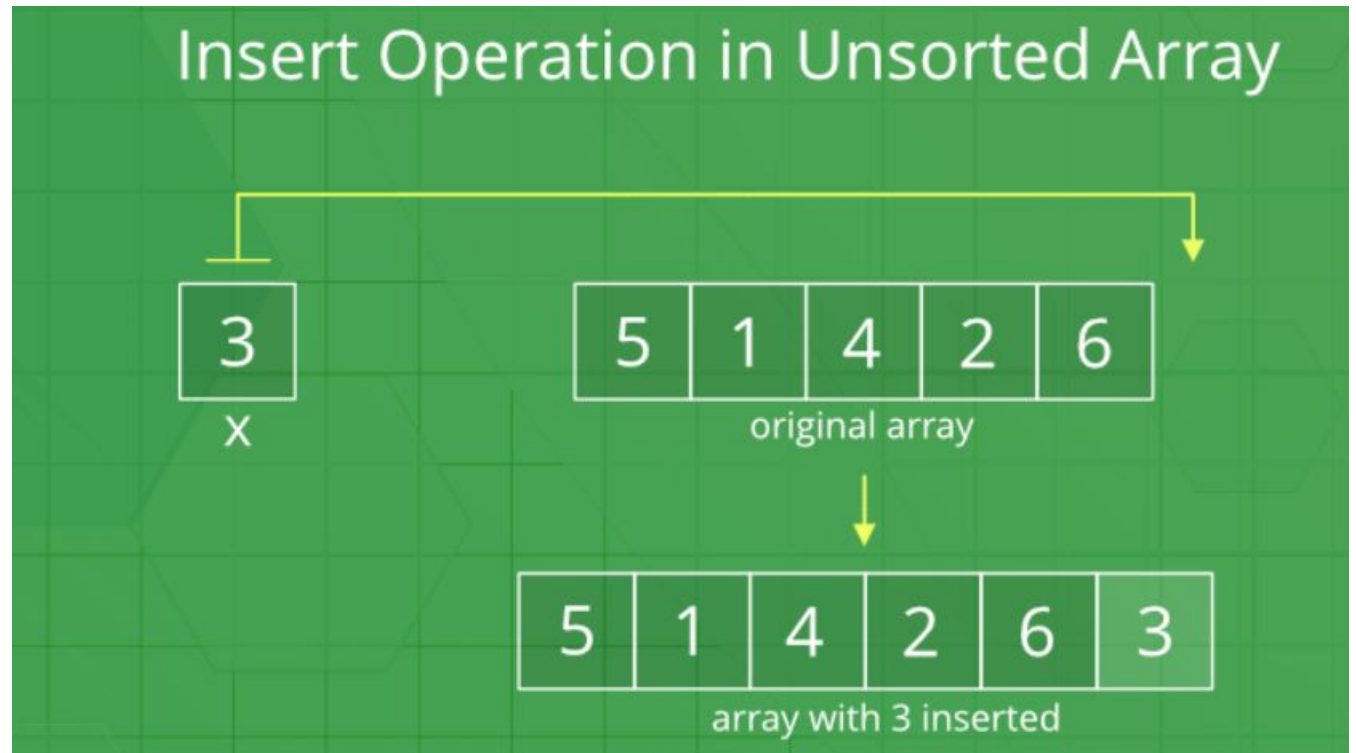
Copying Existing Elements: The loop iterates over each element of the input array **arr** and copies it to the new array **newArr**. This loop runs **n** times, where **n** is the length of the input array **arr**.

Copying existing elements takes **linear time, $O(n)$** .

Overall, the dominant factor in time complexity is the loop that copies existing elements, resulting in a time complexity of **$O(n)$** .

insertion

- In an **unsorted** array, the insert operation is faster as compared to a sorted array because we don't have to care about the position at which the element is to be placed.



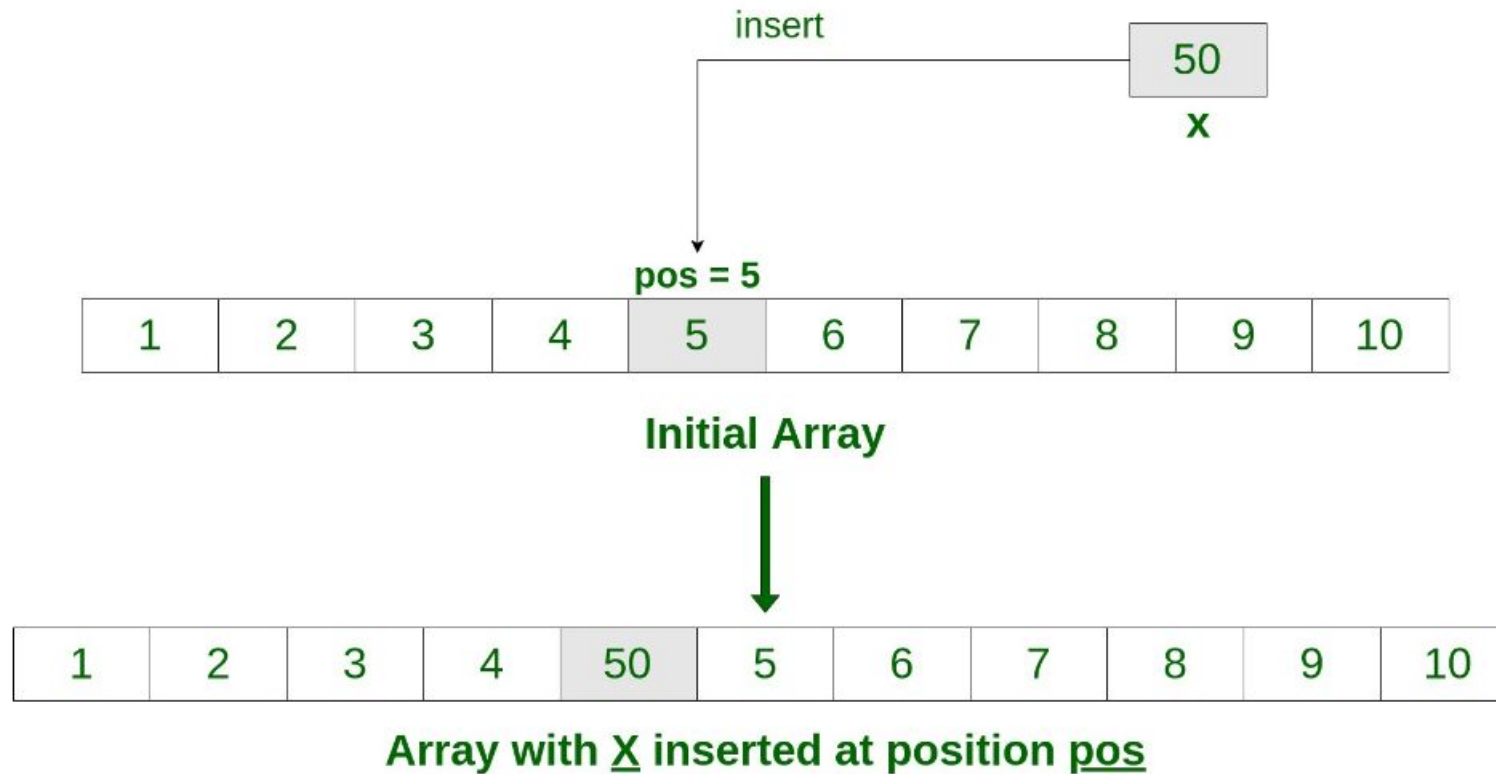


Insert at the end

```
void insertElementAtEnd(int[] array, int value) {  
    if (array.length > 0) {  
        array[array.length - 1] = value;  
    }  
    else {  
        System.out.println("Array is empty.");  
    }  
}
```

Insert at any position

Insert an element at a specific position in an Array.

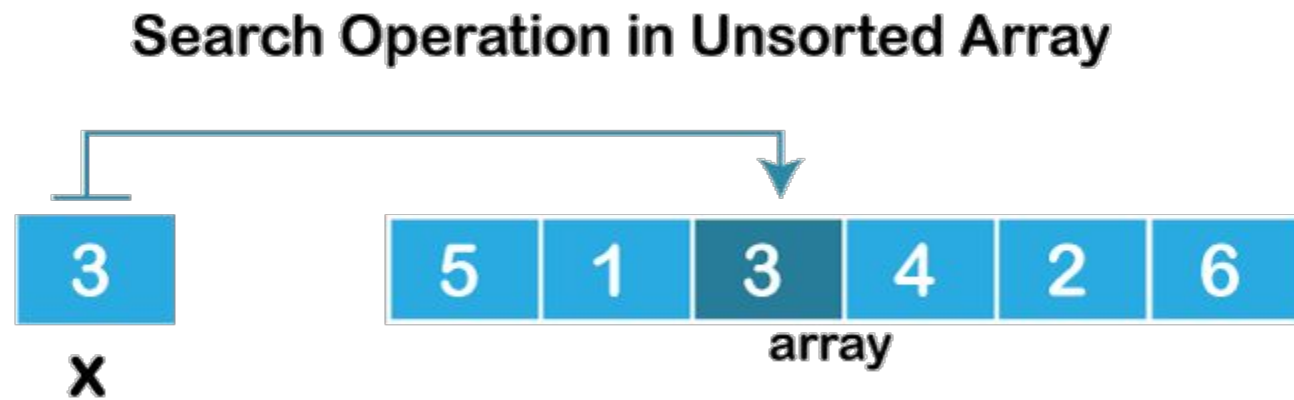




```
void insertElementAtIndex(int[] array, int index, int value) {
    if (index >= 0 && index <= array.length) {
        // Shift elements to the right to make space for the new element
        for (int i = array.length - 1; i > index; i--) {
            array[i] = array[i - 1];
        }
        array[index] = value; //Insert the new element at the specified index
    }
    else {
        System.out.println("Invalid index for insertion.");
    }
}
```

Searching

- With an unsorted array, the search operation can be accomplished by doing a linear traversal from the first to the final element.





```
int findElement(int array[], int key) {  
    for(int i = 0; i < array.length; i++) {  
        if(array[i] == key)  
            return i;  
    }  
    return -1;  
}
```



Deletion and searching

- Home Work

<https://www.geeksforgeeks.org/search-insert-and-delete-in-an-unsorted-array/>



Why not use arrays for everything?

- Arrays seem to get the job done, so why not use them for all data storage? We've already seen some of their disadvantages. In an unordered array you can insert items quickly, in $O(1)$ time, but searching takes slow $O(N)$ time. In an ordered array you can search quickly, in $O(\log N)$ time, but insertion takes $O(N)$ time.
- For both kinds of arrays, deletion takes $O(N)$ time because half the items (on the average) must be moved to fill in the hole. It would be nice if there were data structures that could do everything—insertion, deletion, and searching—quickly, ideally in $O(1)$ time, but if not that, then in $O(\log N)$ time.



References

- <https://www.javatpoint.com/abstract-class-in-java>
- https://www.tutorialspoint.com/data_structures_algorithms/dsa_queue.htm
- <https://www.geeksforgeeks.org/java-program-for-heap-sort/>
- <https://www.geeksforgeeks.org/linked-list-in-java/>
- <https://www.programiz.com/dsa/queue>
- <https://www.javatpoint.com/tree>
- <https://www.edureka.co/blog/java-binary-tree>

